Learning Objectives

Learners will be able to…

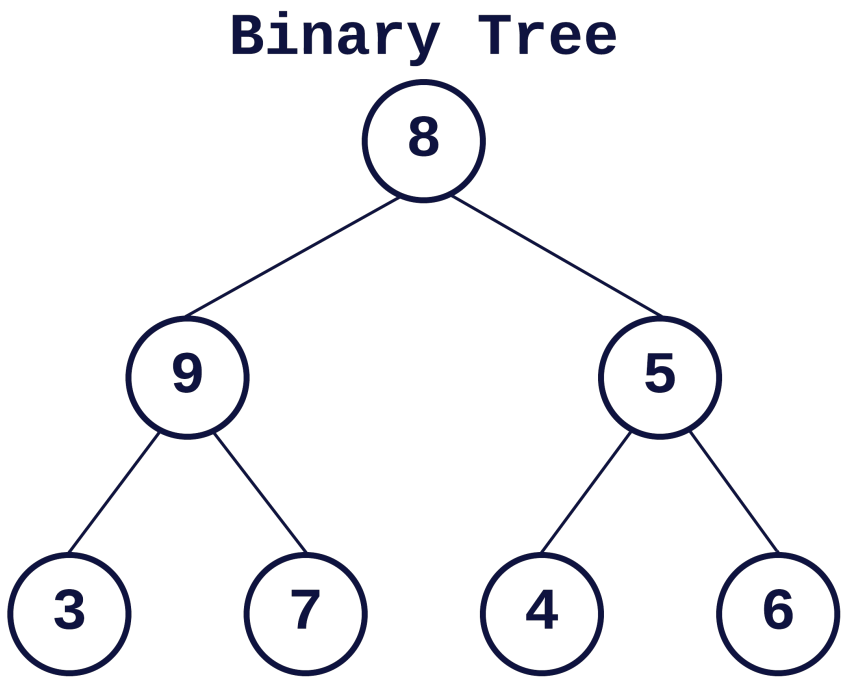
- **Define binary and binary search trees**
- **Differentiate binary and binary search trees**
- **Insert and delete nodes in a binary search tree**
- **Traverse a binary search tree in sequential order**
- **Analyze binary search tree efficiencies**

Trees with Only Two Nodes

In computer science, particularly in the field of data structures, trees represent a hierarchical structure of nodes, where each node links to several subtrees. While trees come in various forms, this assignment focuses on a special category of trees that have, at most, two nodes. These trees are known as **binary trees**. There is even a particular type of binary tree called **binary search trees (BST)**.

This initial exploration into binary trees and BSTs lays the groundwork for understanding how these structures function and how they are used in computer science.

Binary Trees

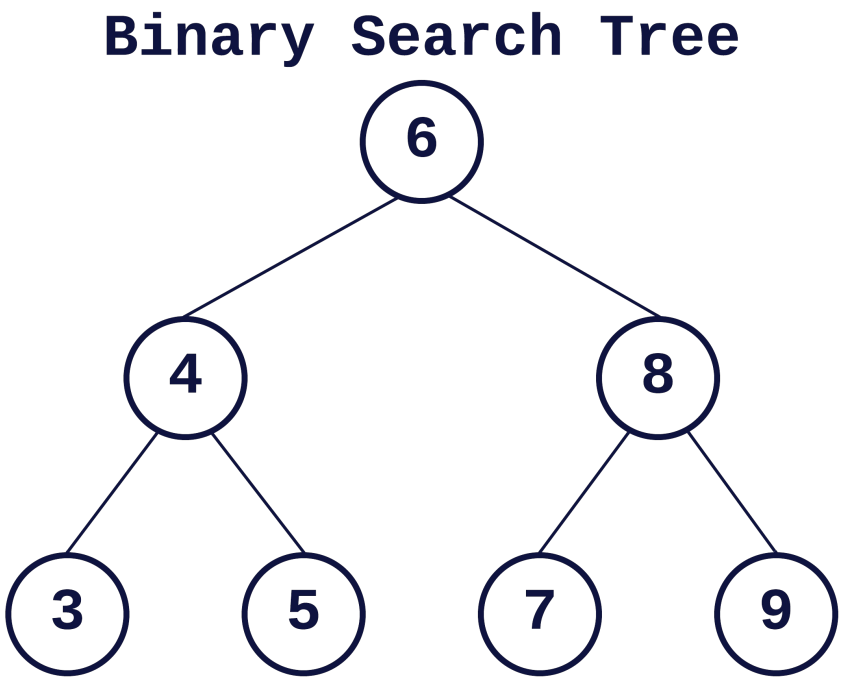


A **binary tree** is a tree data structure in which each node has at most two children, referred to as the left child and the right child. This definition is crucial because it sets binary trees apart from other tree structures that can have any number of children.

There are also no restrictions on the order of data stored in a binary tree. This gives the data structure some added flexibility. Binary trees often serve as the foundation for more complex data structures binary search

trees, heaps (covered later), and segment trees (covered later). They are also used by applications that naturally structure and manage data in a hierarchical manner.

Binary Search Trees



A **binary search tree (BST)** is an ordered binary tree designed for quick searching, insertion, and deletion operations. In a BST, the nodes are arranged following specific properties. For every node, the value of all the nodes in its left subtree is less than its own value, and the value of all the nodes in its right subtree is greater. A BST typically does not contain duplicate values. This property helps maintain a consistent order and enables unique paths to each node.

These properties allow efficient lookup, insertion, and deletion operations. The time complexity of these operations is $O(h)$, where h is the height of the tree. If you perform an in-order traversal, you will retrieve the nodes in sorted order, which is a useful property for various applications. BSTs are excellent for cases where the data is constantly changing, such as managing an online marketplace's product listings or a real-time priority queue.

One of the critical challenges with BSTs is maintaining balance. A balanced tree is one in which the difference in height between the left subtree and right subtree of every node is no greater than one. This allows operations to be performed in $O(logn)$ time. However, without balance (in the case of a degenerate tree), the performance deteriorates to $O(n)$, equivalent to a linked list.

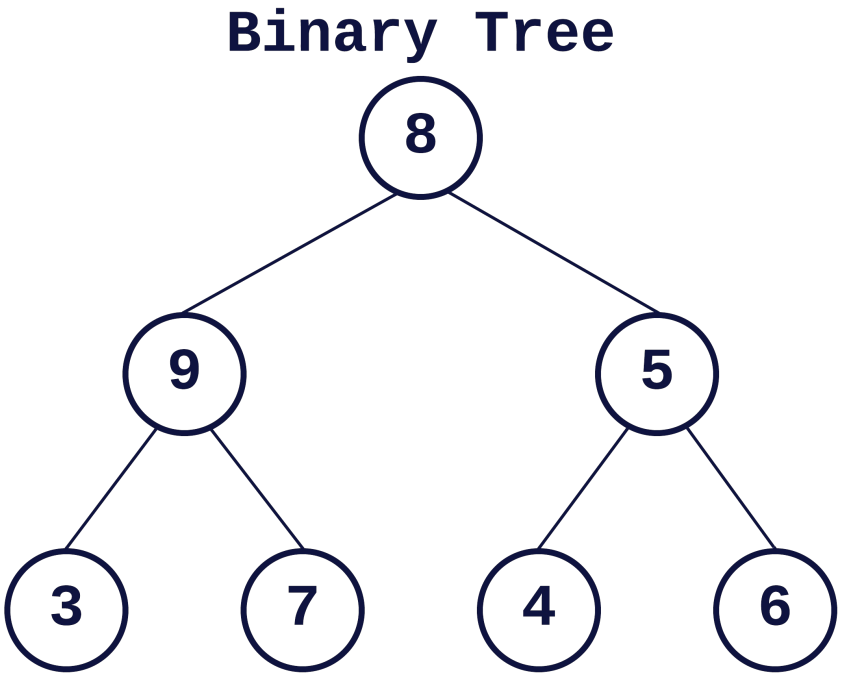
Binary Trees

Deep Dive into Binary Trees

In the world of data structures, binary trees hold a special place due to their efficiency in performing various operations.. Understanding the nuances of binary trees is crucial before we explore more complex tree structures.

What Makes a Tree Binary?

A binary tree is characterized by a specific rule: each node can have zero, one, or two children, and no more. Each child is referred to as a left child or a right child. This rule creates the structure that we recognize as a binary tree.



In the tree above, each node has at most two children. This structure is what defines it as a binary tree. Importantly, there are no restrictions on how the data is stored, which gives the data structure some added flexibility. Binary trees often serve as the foundation for more complex data structures binary search trees, heaps (covered later), and segment trees (covered later). They are also used by applications that naturally structure and manage data in a hierarchical manner.

Properties of Binary Trees

Several important properties derive from the binary tree structure, affecting how we interact with and utilize binary trees:

- **Maximum Number of Nodes:** At any given level i in a binary tree, the maximum number of nodes is 2^i, where i is the depth of the level (assuming the root is at level 0).
- **Maximum Total Nodes:** A binary tree of height h can have at most $2^{(h+1)} - 1$ nodes, where h is the height of the tree (the number of nodes along the longest path from the root node down to the farthest leaf node).
- **Minimum Possible Height:** For a binary tree with n nodes, the minimum possible height or the minimum number of levels is $Log2(n + 1)$.
- **Balanced Binary Tree:** A balanced binary tree is one where the depth of the left and right subtrees of every node differ by at most one. Balanced trees are essential for maintaining optimal performance ($O(logn)$ time complexity) for core operations.
- **Complete Binary Tree:** A complete binary tree is a binary tree in which every level, except possibly the last, is completely filled, and all nodes are as far left as possible.

Types of Binary Trees

There are specific categories of binary trees used for different purposes, including:

- **Full Binary Tree:** Every node has exactly 0 or 2 children (not 1).
- **Perfect Binary Tree:** A full binary tree where all leaf nodes are at the same level.
- **Degenerate (or Pathological) Tree:** A tree where each parent node has only one child, making its performance similar to a linked list.

Understanding these properties and types is crucial for recognizing how binary trees function and are used in algorithms and data processing.

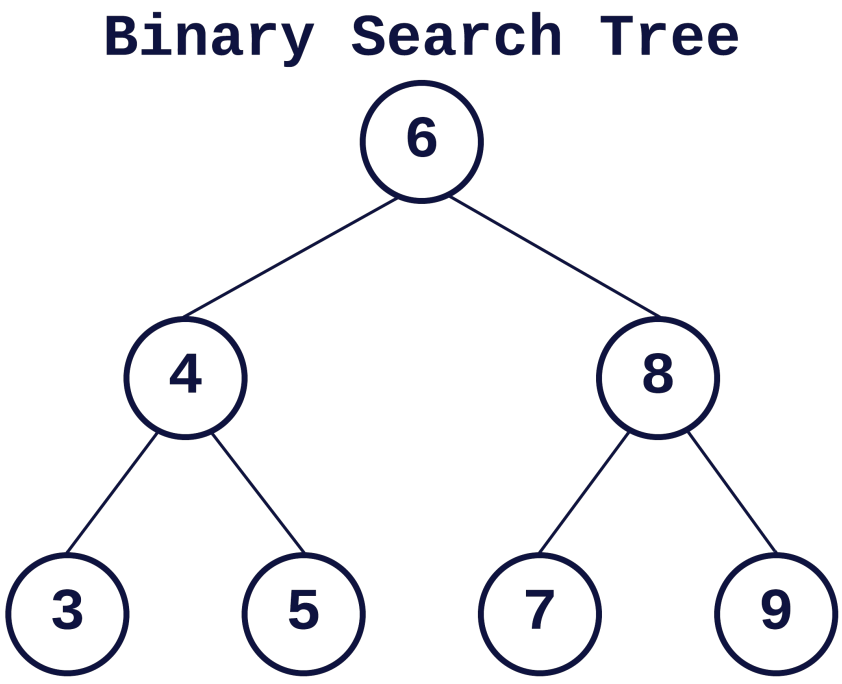
Binary Search Trees

What is a Binary Search Tree?

Binary trees come in various forms, each with its unique properties and uses. One of the most powerful and commonly used types is the **binary search tree (BST)**. It not only inherits the structural properties of a binary tree but also introduces a new set of rules that offer efficient search capabilities. A binary search tree is a node-based binary tree data structure that has the following properties:

- The left subtree of a node contains only nodes with values lesser than the node's value.
- The right subtree of a node contains only nodes with values greater than the node's value.
- There are no duplicate nodes; the left and right subtree each must also be a binary search tree.

These properties significantly reduce the time it takes to locate a specific node because one can use the principles of binary search.



Each value to the left of root node is less than 6, while each value to the right is greater. This rule applies to every node in the tree, not just the root.

Why Use Binary Search Trees?

- **Efficient Search Operations:** Due to their structure, BSTs allow for efficient search operations, similar to those in sorted arrays but with faster insertion and deletion capabilities.
- **Sorted Order Retrieval:** BSTs can retrieve the elements in sorted order using in-order traversal. This feature is beneficial for algorithms that require sorted elements without needing to sort an entire array or list.
- **Flexibility in Size:** Unlike static data structures (like arrays), BSTs are dynamic and can easily expand or shrink in size, accommodating new elements or removing existing ones.
- **Balanced BSTs:** Certain types of BSTs (like AVL trees or Red-Black trees) self-balance, ensuring that the tree remains proportioned and that operations like insertion, deletion, and search take logarithmic time.

Potential Downsides

However, there are certain limitations and considerations as well:

- **Complexity:** Implementing BSTs is generally more complex than other linear data structures due to pointer usage and recursive thinking during operations.
- **Unbalanced Trees:** If not maintained, BSTs can become skewed or unbalanced, especially after numerous insertion and deletion operations. This imbalance can lead to decreased performance, with time complexities degrading from $O(logn)$ to $O(n)$ in the worst case.

Real-World Applications

BSTs are instrumental in various applications, including:

- Database indexing, where quick search, insertion, and deletion are crucial.
- Dynamic sorting of datasets that frequently change.
- Implementing associative arrays (maps) efficiently.

Insertion in Binary Search Trees

Inserting Nodes into a Binary Search Tree

Understanding the theory behind BSTs is integral, but nothing beats actual implementation. Let's construct a simple BST with operations for inserting and searching nodes. We'll create a robust structure that can easily be expanded with additional functionalities.

The first thing we need to do is create a struct for the nodes in our tree. Each node has three attributes: the value it holds, a pointer to the left child, and a pointer to the right child. In the constructor (not explicitly shown in C++), set the pointers to `nullptr` and store the given value in the `value` attribute.

```
struct TreeNode {
    int value;
    TreeNode* left;
    TreeNode* right;

    // Constructor
    TreeNode(int value) : value(value), left(nullptr),
        right(nullptr) {}
};
```

Now, we'll create the `BinarySearchTree` class, which will encapsulate our tree operations. The class has only one attribute, `root`, which is the pointer to the first node in our tree. In the constructor, set `root` to `nullptr`.

```
class BinarySearchTree {
private:
    TreeNode* root; // The root node of our BST

public:
    // Constructor
    BinarySearchTree() : root(nullptr) {
        // Start with an empty tree
    }
};
```

Similar to inserting nodes in the tree, we will use recursion for inserting nodes. Start with the `insert` method that takes a value to insert into the tree. Set the root attribute to the result of the overloaded method `insert`, which takes the root of the tree and the value to `insert`. Additionally, to access the root node externally, we have included a `getRoot()` method that

returns a pointer to the `root` node. This method allows external code, such as the main method, to access and display information about the root and its child nodes in the tree structure.

```
public:
    BinarySearchTree() : root(nullptr) {}

    void insert(int value) {
        root = insert(root, value);
    }

    // Public method to get the root (added for demonstration)
    TreeNode* getRoot() const {
        return root;
    }

private:
    TreeNode* insert(TreeNode* node, int value) {

        if (node == nullptr) {
            return new TreeNode(value);
        }

        if (value < node->value) {
            node->left = insert(node->left, value);
        } else if (value > node->value) {
            node->right = insert(node->right, value);
        }

        return node;
    }
};
```

Create the main method to instantiate a `BinarySearchTree` object. Insert nodes with the values `6` (this will be the root), `4`, `8`, `2`, and `5`. Nodes `4` and `8` are the children (left and right respectively) of the root. Nodes `2` and `5` are the children (left and right respectively) of node `4`.

```
int main() {
    BinarySearchTree bst;

    // Inserting values
    bst.insert(6);
    bst.insert(4);
    bst.insert(8);
    bst.insert(2);
    bst.insert(5);

    // Print values in the tree (similar output logic as in C++)
    // Note: C++ cout and endl are used for output
    std::cout << "Root: " << bst.getRoot()->value << std::endl;
    std::cout << "Left child: " << bst.getRoot()->left->value <<
        std::endl;
    std::cout << "Right child: " << bst.getRoot()->right->value
        << std::endl;
    std::cout << "Left child of Node 4: " << bst.getRoot()-
        >left->left->value << std::endl;
    std::cout << "Right child of Node 4: " << bst.getRoot()-
        >left->right->value << std::endl;

    return 0;
}
```

▼ **Code**

Your C++ code should look like this:

```
struct TreeNode {
    int value;
    TreeNode* left;
    TreeNode* right;

    // Constructor
    TreeNode(int value) : value(value), left(nullptr),
        right(nullptr) {}
};

class BinarySearchTree {
private:
    TreeNode* root; // The root node of our BST

public:
    BinarySearchTree() : root(nullptr) {}

    void insert(int value) {
        root = insert(root, value);
    }

    // Public method to get the root (added for demonstration)
    TreeNode* getRoot() const {
        return root;
    }

private:
    TreeNode* insert(TreeNode* node, int value) {
```

```
        if (node == nullptr) {
            return new TreeNode(value);
        }

        if (value < node->value) {
            node->left = insert(node->left, value);
        } else if (value > node->value) {
            node->right = insert(node->right, value);
        }

        return node;
    }
};

int main() {
    BinarySearchTree bst;

    // Inserting values
    bst.insert(6);
    bst.insert(4);
    bst.insert(8);
    bst.insert(2);
    bst.insert(5);

    // Print values in the tree (similar output logic as in C++)
    std::cout << "Root: " << bst.getRoot()->value << std::endl;
    std::cout << "Left child: " << bst.getRoot()->left->value <<
        std::endl;
    std::cout << "Right child: " << bst.getRoot()->right->value
        << std::endl;
    std::cout << "Left child of Node 4: " << bst.getRoot()-
        >left->left->value << std::endl;
    std::cout << "Right child of Node 4: " << bst.getRoot()-
        >left->right->value << std::endl;

    return 0;
}
```

You should see the following output when you run the C++ code:

```
Root: 6
Left child: 4
Right child: 8
Left child of Node 4: 2
Right child of Node 4: 5
```

Now that we have an understanding on how to insert a node into a binary search tree, we are going to move on to something a bit more challenging – deleting a node from a binary search tree.

Deletion in Binary Search Trees

Removing Nodes from a Binary Search Tree

We will now delve into one of the more complex operations on a binary search tree deletion. Deleting nodes in a BST requires a careful approach that ensures the binary search tree property holds after the deletion. Unlike linear data structures like arrays or linked lists, we cannot simply remove a node; we must consider the node's children and how to "re-link" the tree.

Remember, we must maintain the BST property after deletion. In other words, the following must still hold true: A binary tree with nodes arranged such that left children have smaller values, right children have larger values, and no duplicate nodes exist. Both left and right subtrees must also be BSTs.

Three possible scenarios arise when you want to delete a node from a BST:

1. **Node with No Children (leaf node):** This is the simplest case; we can simply remove the node from the tree.
2. **Node with One Child:** We can delete the node and replace it with its child.
3. **Node with Two Children:** This is the most complex case. We will find the node's in-order successor (the smallest node in the right subtree) or in-order predecessor (the largest node in the left subtree) and replace the node's value with it. Then, we delete the in-order successor or predecessor (which has at most one child).

Implementation

Let's update our `BinarySearchTree` class from the previous page to include a method for deleting nodes. To do this, we'll also need a helper method to find the minimum value in a tree (useful for finding the in-order successor). First, add the method to find the node with the minimum value.

```
    TreeNode* minValNode(TreeNode* node) {
        // Find the leftmost leaf
        while (node->left != nullptr) {
            node = node->left;
        }
        return node;
    }
```

Now we can implement the `deleteNode` method. This will be a helper method that takes the value to remove. The method then calls the recursive method to remove the value.

```
    void deleteNode(int value) {
        root = deleteNode(root, value);
    }
```

The `deleteNode` method handles the actual deletion of the node. It takes two parameters: the current node and the value to be removed. Start by creating the base case, which is when the current node is `nullptr`. Return `node` when the base case is satisfied.

```
    TreeNode* deleteNode(TreeNode* node, int value) {
        // If the tree is empty, return the same node
        if (node == nullptr) return node;

    }
```

If the current node is not `nullptr`, iterate over the tree. Start by recursing down the left subtree if the given value is less than the current value. Otherwise, recurse down the right subtree if the given value is greater than the current value.

```
    TreeNode* deleteNode(TreeNode* node, int value) {
        // If the tree is empty, return the same node
        if (node == nullptr) return node;

        // Otherwise, recurse down the tree
        if (value < node->value) {
            node->left = deleteNode(node->left, value);
        } else if (value > node->value) {
            node->right = deleteNode(node->right, value);
        }

    }
```

The two conditionals above account for when the given value is less than or greater than the value of the current node. When the two values are equal, we need to account for the three possible cases above. If the target node does not have children, we simply remove it by changing the respective parent pointer to `nullptr`. If the target node has one child, we link its parent to its child, and then delete the node.

```
TreeNode* deleteNode(TreeNode* node, int value) {
    // If the tree is empty, return the same node
    if (node == nullptr) return node;

    // Otherwise, recurse down the tree
    if (value < node->value) {
        node->left = deleteNode(node->left, value);
    } else if (value > node->value) {
        node->right = deleteNode(node->right, value);
    } else {
        // Node with only one child or no child
        if (node->left == nullptr) {
            return node->right;
        } else if (node->right == nullptr) {
            return node->left;
        }

    }
}
```

However, if the target node has two children, we need to find its in-order successor. This is done with the `minValNode` method. Then copy the successor's value to the node, and then recursively delete the successor's node. Lastly, return the node.

```
TreeNode* deleteNode(TreeNode* node, int value) {
    // If the tree is empty, return the same node
    if (node == nullptr) return node;

    // Otherwise, recurse down the tree
    if (value < node->value) {
        node->left = deleteNode(node->left, value);
    } else if (value > node->value) {
        node->right = deleteNode(node->right, value);
    } else {
        // Node with only one child or no child
        if (node->left == nullptr) {
            return node->right;
        } else if (node->right == nullptr) {
            return node->left;
        }

        // Node with two children: Get the in-order
    successor (smallest in the right subtree)
        node->value = minValNode(node->right)->value;

        // Delete the in-order successor
        node->right = deleteNode(node->right, node->value);
    }

    return node;
}
```

Update the `main` method so that node `4` is removed from the tree. The root's left child should now become `5`, and `5` should have `2` as the left child.

```
int main() {
    BinarySearchTree bst;

    // Inserting values
    bst.insert(6);
    bst.insert(4);
    bst.insert(8);
    bst.insert(2);
    bst.insert(5);

    // Delete node
    bst.deleteNode(4);

    // Print values in the tree
    std::cout << "Root: " << bst.getRoot()->value << std::endl;
    std::cout << "Left child: " << bst.getRoot()->left->value <<
        std::endl;
    std::cout << "Right child: " << bst.getRoot()->right->value
        << std::endl;
    std::cout << "Left child of Node 5: " << bst.getRoot()-
        >left->left->value << std::endl;

    return 0;
}
```

▼ **Code**

Your code should look like this:

```
struct TreeNode {
    int value;
    TreeNode* left;
    TreeNode* right;

    TreeNode(int value) : value(value), left(nullptr),
        right(nullptr) {}
};

class BinarySearchTree {
private:
    TreeNode* root;

public:
    BinarySearchTree() : root(nullptr) {}

    void insert(int value) {
        root = insert(root, value);
    }

    void deleteNode(int value) {
        root = deleteNode(root, value);
    }

    TreeNode* getRoot() const {
```

```
        return root;
    }

private:
    TreeNode* minValNode(TreeNode* node) {
        // Find the leftmost leaf
        while (node->left != nullptr) {
            node = node->left;
        }
        return node;
    }

    TreeNode* deleteNode(TreeNode* node, int value) {
        // If the tree is empty, return the same node
        if (node == nullptr) return node;
        // Otherwise, recurse down the tree
        if (value < node->value) {
            node->left = deleteNode(node->left, value);
        } else if (value > node->value) {
            node->right = deleteNode(node->right, value);
        } else {
            // Node with only one child or no child
            if (node->left == nullptr) {
                return node->right;
            } else if (node->right == nullptr) {
                return node->left;
            }
            // Node with two children: Get the in-order
        successor (smallest in the right subtree)
            node->value = minValNode(node->right)->value;
            // Delete the in-order successor
            node->right = deleteNode(node->right, node->value);
        }
        return node;
    }

    TreeNode* insert(TreeNode* node, int value) {
        if (node == nullptr) {
            node = new TreeNode(value);
            return node;
        }
        if (value < node->value) {
            node->left = insert(node->left, value);
        } else if (value > node->value) {
            node->right = insert(node->right, value);
        }
        return node;
    }
};

// Main method to demonstrate insertion and deletion
int main() {
    BinarySearchTree bst;
    // Inserting values
    bst.insert(6);
    bst.insert(4);
    bst.insert(8);
    bst.insert(2);
```

```
    bst.insert(5);
    bst.deleteNode(4);
    // Note: C++ cout and endl are used for output
    std::cout << "Root: " << bst.getRoot()->value << std::endl;
    std::cout << "Left child: " << bst.getRoot()->left->value <<
        std::endl;
    std::cout << "Right child: " << bst.getRoot()->right->value
        << std::endl;
    std::cout << "Left child of Node 5: " << bst.getRoot()-
        >left->left->value << std::endl;
    // std::cout << "Right child of Node 5: " << bst.getRoot()-
        >left->right->value << std::endl;
    return 0;
}
```

You should see the following output:

```
Root: 6
Left child: 5
Right child: 8
Left child of Node 5: 2
```

Next, we are going to talk about traversing a binary search tree.

Traversal

Printing a Tree

We have already talked about different traversal strategies in a previous assignment. However, the ordered property of binary search trees allow you to print a tree in sequential order (smallest to largest in our example) when you use an in-order traversal. Recall that an in-order traversal iterates over a tree in the following manner:

1. Traverse the left subtree by recursively calling the in-order method.
2. Display the data part of the root (or current node).
3. Traverse the right subtree by recursively calling the in-order method.

Let's enhance our `BinarySearchTree` class by adding an in-order traversal method that prints the elements. Additionally, we'll include a method to visually display the structure of the tree for better understanding. Again, we are going to use method overloading to differentiate the helper method (no parameters) from the recursive method (one parameter). The logic contained in these methods is the same from what we have seen before.

First, add the in-order traversal method:

```
    void inorderTraversal() {
        std::cout << "In-order Traversal:";
        inorder(root);
        std::cout << std::endl;
    }

    void inorder(TreeNode* node) {
        if (node != nullptr) {
            inorder(node->left); // visit left subtree
            std::cout << " " << node->value; // print data of
        the node
            inorder(node->right); // visit right subtree
        }
    }
```

Printing by Level

So far, we have been printing a tree by traversing it node by node. While this does have value, the end result does not give us a representation of the hierarchical structure of the tree. This time, we are going to traverse the tree level by level, printing the corresponding nodes (and whitespace) to give us a better representation of the data structure.

We will start by creating the `printLevels` helper method. It takes no parameters and calls the overloaded `printLevels` method and passes it the root node.

```
    void printLevelOrder() {
        printLevels(root);
    }
```

To traverse the tree by level, we need a way of keeping track of the nodes on each level. We can't use the binary tree itself as it is organized in a top-down manner. Instead, we need an intermediary data structure. We are working with the tree below:

```
    6
   / \
  4   8
 / \
2   5
```

As we progress through the levels, we first need to store `6` and then print it. On the next level, we store `4` and `8` (in that order), and then print `4` and `8` (in that order). Finally, we store `2` and `5` (in that order), and then print `2` and `5` (in that order). Does this sound familiar? The first element stored is the first element printed (FIFO). We need a queue. However, we are not going to build a `queue` ourselves. Instead, we are going to use C++'s built-in data structure. This library has already been included for you.

```
#include <queue>
```

Next, we are going create the overloaded `printLevels` method that takes no parameters. First, check to see if the tree is empty. If so, print a message to the user and exit the method. Otherwise, create a queue and add the root node to it. Then instantiate the variable `level` and set its value to `0`.

```
    void printLevels(TreeNode* node) {
        if (node == nullptr) {
            std::cout << "The tree is empty" << std::endl;
            return;
        }

        std::queue<TreeNode*> queue;
        queue.push(node);

        int level = 0;
    }
```

Use a while loop to iterate over the queue as long as it is not empty. Create the variable `size` and set its value to the size of the queue. Print some text that tells the user what level they are viewing, and then increment `level` by

one.

```
    void printLevels(TreeNode* node) {
        if (node == nullptr) {
            std::cout << "The tree is empty" << std::endl;
            return;
        }

        std::queue<TreeNode*> queue;
        queue.push(node);

        int level = 0;

        while (!queue.empty()) {
            int size = queue.size();
            std::cout << "Level " << level << ": ";
            level++;
        }
    }
```

Create a for loop that runs as many times as the size of the queue. For each iteration, remove the first element (a node) from the queue and store it in a temporary variable. Print the value of the node followed by a space. If there is a left child to the current node, add it to the queue. Do the same thing for the right child. Lastly, use a `cout` statement to terminate the current line of output and go to the next line.

```
    void printLevels(TreeNode* node) {
        if (node == nullptr) {
            std::cout << "The tree is empty" << std::endl;
            return;
        }

        std::queue<TreeNode*> queue;
        queue.push(node);

        int level = 0;

        while (!queue.empty()) {
            int size = queue.size();
            std::cout << "Level " << level << ": ";
            level++;

            // traverse all nodes of current level
            for (int i = 0; i < size; i++) {
                TreeNode* currentNode = queue.front();
                queue.pop();

                // print current node
                std::cout << currentNode->value << " ";

                // add child nodes to queue for next level
                if (currentNode->left != nullptr)
        queue.push(currentNode->left);
                if (currentNode->right != nullptr)
        queue.push(currentNode->right);
            }

            // newline after printing each level
            std::cout << std::endl;
        }
    }
```

Update the `main` method to remove the deletion and print statements from the last page. In their place, we are going to call the `inorderTraversal` and `printLevelOrder` methods.

```
int main() {
    BinarySearchTree bst;

    // Inserting values
    bst.insert(6);
    bst.insert(4);
    bst.insert(8);
    bst.insert(2);
    bst.insert(5);

    // Print tree
    bst.inorderTraversal();
    bst.printLevelOrder();

    return 0;
}
```

▼ **Code**

Your code should look like this:

```
struct TreeNode {
    int value;
    TreeNode* left;
    TreeNode* right;

    // Constructor
    TreeNode(int value) : value(value), left(nullptr),
        right(nullptr) {}
};

class BinarySearchTree {
private:
    TreeNode* root;

    void inorder(TreeNode* node) {
        if (node != nullptr) {
            inorder(node->left); // visit left subtree
            std::cout << " " << node->value; // print data of
        the node
            inorder(node->right); // visit right subtree
        }
    }

    void printLevels(TreeNode* node) {
        if (node == nullptr) {
            std::cout << "The tree is empty" << std::endl;
            return;
        }

        std::queue<TreeNode*> queue;
        queue.push(node);
        int level = 0;

        while (!queue.empty()) {
            int size = queue.size();
```

```
            std::cout << "Level " << level << ": ";
            level++;

            // traverse all nodes of current level
            for (int i = 0; i < size; i++) {
                TreeNode* currentNode = queue.front();
                queue.pop();

                // print current node
                std::cout << currentNode->value << " ";

                // add child nodes to queue for next level
                if (currentNode->left != nullptr)
        queue.push(currentNode->left);
                if (currentNode->right != nullptr)
        queue.push(currentNode->right);
            }

            // newline after printing each level
            std::cout << std::endl;
        }
    }

public:
    // Constructor
    BinarySearchTree() : root(nullptr) {}

    // Insert a node (public method)
    void insert(int value) {
        root = insert(root, value); // Call the recursive helper
        method to insert
    }

    // Inorder traversal (public method)
    void inorderTraversal() {
        std::cout << "In-order Traversal:";
        inorder(root);
        std::cout << std::endl;
    }

    // Level-order traversal (public method)
    void printLevelOrder() {
        printLevels(root);
    }

    TreeNode* insert(TreeNode* node, int value) {
        // If the tree is empty, return a new node
        if (node == nullptr) {
            node = new TreeNode(value);
            return node;
        }

        // Otherwise, recur down the tree
        if (value < node->value) {
            node->left = insert(node->left, value);
        } else if (value > node->value) {
            node->right = insert(node->right, value);
        }
```

```
        // return the (unchanged) node pointer
        return node;
    }
};

int main() {
    BinarySearchTree bst;

    // Inserting values
    bst.insert(6);
    bst.insert(4);
    bst.insert(8);
    bst.insert(2);
    bst.insert(5);

    // Inorder traversal
    bst.inorderTraversal();

    // Level-order traversal
    bst.printLevelOrder();

    return 0;
}
```

You should see the following output:

```
In-order Traversal: 2 4 5 6 8
Level 0: 6
Level 1: 4 8
Level 2: 2 5
```

Formative Assessment 1

Formative Assessment 2